

JOSÉ ANTONIO FUENTES SANTIAGO

GUÍA DE PROGRAMACIÓN EN UN APOCALIPSIS ZOMBI



Single Player

Software Development Book

Guía de Programación en un Apocalipsis Zombi (Demo)

José Antonio Fuentes Santiago

Índice

Bloque 1. Fundamentos de programación	1
Capítulo 1. Los comentarios. Anotando lo que necesitas	3
Comentarios de una línea	3
Comentarios multi-línea	3
Comentarios JSDoc	4
Desactivar código con comentarios	4
Qué merece un comentario	4
Ejercicios de supervivencia	5
Capítulo 2. Variables y constantes. Organizando tu mochila	7
Variables	7
Constantes	8
Buenas prácticas para un código limpio	9
Evita los números mágicos	10
Ejercicios de supervivencia	11
Capítulo 3. Tipos de datos primitivos. Separando los suministros	13
Tipos numéricos	13
Cadenas	15
Booleanos	16
Cuando no tenemos recursos: <code>null</code> y <code>undefined</code>	17
Unión de tipos	18
El tipo <code>any</code>	18
El tipo <code>unknown</code>	20
Otros tipos primitivos	20
Ejercicios de supervivencia	20
Capítulo 4. Operadores. Cómo leer las señales del entorno	22
Operadores aritméticos	22
Operadores de cadena	23
Operadores de comparación	24
Operadores lógicos	26
Operadores unarios	30
Asignación compuesta	31
Ejercicios de supervivencia	32

Continuará...**36**

Bloque 1. Fundamentos de programación

Ahora...

Primero, fue un aviso en los móviles.

Luego, los gritos en la calle.

Después, el silencio.

Tras semanas de rumores, bastaron unas horas para que el mundo que conocías dejara de funcionar. Las calles están vacías, los teléfonos no suenan, y algo se mueve al otro lado de la ciudad... algo que no camina como tú. Algo que no piensa como tú. Algo que nunca se detiene...

El aviso era claro, había que abandonar las zonas más pobladas y buscar un refugio lejos de las multitudes.

Aún estás en tu casa. Echas un vistazo a tu alrededor y ves comida enlatada, tus viejos apuntes, una linterna medio descargada y un portátil antiguo que, milagrosamente, sigue funcionando. Es ahora o nunca. Vas a necesitar conocimiento, ingenio, y una mochila bien preparada.

No sabes cuánto tiempo estarás ahí fuera. Pero sabes una cosa: sobrevivirás. Y para eso, necesitas entender cómo funciona este nuevo mundo.

Hoy, aprenderás a revisar cada rincón de tu mochila: desde cómo anotar lo importante, clasificar los suministros y organizar tus recursos, hasta interpretar señales del entorno y tomar tus primeras decisiones difíciles. Incluso habrá tiempo para reagrupar tus herramientas y dar los últimos retoques antes de salir por la puerta.

En este bloque...

Verás cómo puedes dejar preparada la mochila para abandonar la seguridad de tu casa... y enfrentarte al mundo exterior.

- **Capítulo 1. Los comentarios:** Aprende a documentar tu código como si fuera tu diario de supervivencia.
- **Capítulo 2. Variables y constantes:** Descubre cómo almacenar y organizar información en tu programa.
- **Capítulo 3. Tipos de datos primitivos:** Conoce los diferentes tipos de datos básicos y cómo usarlos.
- **Capítulo 4. Operadores:** Domina las operaciones básicas para manipular datos y tomar decisiones.
- **Capítulo 5. Condicionales:** Implementa la lógica para tomar decisiones en tu código.
- **Capítulo 6. Bucles:** Aprende a repetir acciones de forma eficiente.
- **Desafío 1: Simulador de supervivencia de 7 días:** Pon a prueba todo lo aprendido.
- **Resumen del bloque 1:** Repasa los conceptos clave.

Capítulo 1. Los comentarios. Anotando lo que necesitas

Antes de salir al exterior y enfrentarte a las criaturas que acechan en cada rincón de la ciudad, cualquier superviviente con un poco de sensatez prepara su libreta y comienza a escribir información vital: rutas de escape, cómo funcionan las herramientas que podrían salvarte la vida, patrones de comportamiento zombi que has observado, etc. Cada anotación podría marcar la diferencia entre sobrevivir o convertirte en una de esas cosas.

Los **comentarios** son pequeñas notas que te ayudan a saber qué hace cada parte del código, como si apuntaras recordatorios en la tapa de tu libreta de campo. Éstos, son ignorados por el transpilador, intérprete o compilador (según el tipo de lenguaje de programación como vimos antes), por lo que sólo te sirven a ti.

Comentarios de una línea

Los comentarios de una línea comienzan por `//`, por lo que todo lo que escribas hasta el final de ésta pertenecerá al comentario.

```
// Declaramos nuestros recursos iniciales.  
let totalRaciones: number = 24; // Raciones totales disponibles.  
let racionesAlDia: number = 3; // Consumo diario por persona.  
  
// Calcula el número de días con provisiones.  
let diasProvisiones: number = totalRaciones / racionesAlDia;
```

Comentarios multi-línea

Por otro lado, puedes añadir comentarios que ocupen varias líneas. Este tipo de comentarios comienza por `/*` y termina con `*/`. Todo el texto que incluyas entre la apertura y el cierre será tratado como parte del comentario.

```
/*  
 * Raciones necesarias para la huida:  
 * - Cada día consumes 3 raciones.  
 * - Sumamos un día extra por si algo sale mal.  
 */  
let diasHuida: number = 4;  
let racionesHuida: number = (diasHuida + 1) * 3;
```

Comentarios JSDoc

Hay una serie de comentarios especiales que puedes usar con TypeScript y JavaScript, que se muestran como ayuda contextual en la mayoría de editores de código y se exportan a distintos formatos de fichero de documentación. A este tipo de comentarios se les llama JSDoc y siguen una serie de normas que te ayudarán a documentar adecuadamente tu código.

Estos comentarios comienzan por `/**` y terminan con `*/`, como cualquier otro comentario de varias líneas. En TypeScript no hace falta repetir el tipo dentro del comentario: ya está en el código.

```
/**
 * Función encargada de comprobar si una zona es segura.
 * @param coordenadas Coordenadas de la zona a comprobar.
 * @returns Indicador de si la zona es segura o no.
 */
function esZonaSegura(coordenadas: string): boolean {
    // Simulación: las zonas que empiezan por "A" son seguras.
    return coordenadas.startsWith("A");
}
```

Desactivar código con comentarios

Los comentarios también sirven para “apagar” una línea sin borrarla. Es muy útil cuando pruebas algo y quieres poder volver atrás rápido:

```
let racionesDisponibles: number = 12;
// racionesDisponibles = racionesDisponibles - 3;
console.log(racionesDisponibles); // 12: la resta está desactivada.
```

Lo verás muchas veces en este libro, sobre todo para mostrar código que daría error.



Nota en la cabaña

En la mayoría de editores puedes comentar o descomentar las líneas seleccionadas con `Ctrl + /` (`Cmd + /` en macOS).

Qué merece un comentario

Compara estos dos comentarios:

```
let diasViaje: number = 4;  
  
// Sumamos 1 a los días.  
let diasConMargen: number = diasViaje + 1;  
  
// Un día extra por si la ruta está cortada y hay que dar un rodeo.  
let diasPrevistos: number = diasViaje + 1;
```

El primero repite lo que ya dice el código: cualquiera ve que se suma 1. El segundo explica **por qué** se suma, que es justo lo que el código no puede contar. Cuando dudes si escribir un comentario, pregúntate si responde a un “por qué”. Si solo describe el “qué”, probablemente sobra.



Consejo de superviviente

Si tienes todo bien organizado y etiquetado, no necesitas mirar dentro de cada bolsa para saber lo que hay. Lo mismo pasa con la programación: si eliges bien los nombres, las estructuras y el código es lo suficientemente claro, apenas necesitarás comentarios. Sin embargo, hay ocasiones en las que los comentarios también te serán útiles para explicar decisiones complejas o advertir sobre trampas ocultas en el código.

Ejercicios de supervivencia

Ejercicio 1.1: Documentando tu equipo

Llevas horas en tu casa, con las persianas bajadas y el silencio roto solo por gemidos distantes y pasos arrastrados que se acercan lentamente. Afuera, la ciudad ya no es segura: las criaturas rondan por las calles, y pronto tendrás que salir antes de que detecten tu ubicación. Antes de dar ese paso fatal, abres tu libreta de campo: es hora de registrar lo que llevas en la mochila. No basta con empaquetar, necesitas claridad absoluta. Cada objeto cuenta, cada decisión puede marcar la diferencia entre sobrevivir o convertirte en parte de la horda.

Tu misión: crea un archivo TypeScript (recuerda, con la extensión `.ts`) que documente tu equipo de supervivencia:

- Copia las cuatro variables de abajo y escribe encima de cada una un comentario de una línea que explique para qué te sirve ese objeto.

- Al final del archivo, escribe un comentario de varias líneas con tu estrategia para los próximos días.
- La linterna se ha quedado sin pilas: desactiva su línea con un comentario y comprueba que el archivo se sigue ejecutando.

```
let mochila = "Mochila táctica 40L";
```

```
let litrosAgua = 2.5;
```

```
let racionesComida = 8;
```

```
let linterna = true;
```

Capítulo 2. Variables y constantes. Organizando tu mochila

El silencio de la casa se rompe con un gemido lejano que flota en el aire nocturno. No hay tiempo que perder: tu mochila de supervivencia debe estar perfectamente organizada antes de que esos sonidos se acerquen más. Cada recurso que guardes debe llevar una etiqueta clara con su nombre: “agua”, “comida”, “mapa”, “munición”. Así, cuando necesites algo urgentemente mientras huyes, sabrás exactamente dónde encontrarlo sin hacer ruido innecesario.

En programación, esas etiquetas son el nombre de las **variables** y **constantes** que te permiten organizar y acceder a tus recursos de forma eficiente.

Variables

Para comenzar a organizarte, usas tus pósts en los que escribes el nombre de algún suministro de tu mochila. Una variable sería el equivalente a esos pósts: puedes despegarlos y ponerlos en otro objeto cuando lo necesites. Por ejemplo, en una botella tienes el póst “agua”, si la botella se agota podrás poner la etiqueta en otra botella de agua. Es decir, hemos cambiado el contenido, pero la etiqueta sigue siendo útil, siempre y cuando tenga el mismo tipo de dato.

El formato para declarar una variable es el siguiente:

```
// El símbolo "=" asigna un valor a la variable.  
let nombreVariable: tipo = valor;  
  
// Puedes declararla sin valor inicial, pero TypeScript no te dejará  
// usarla hasta que le asignes uno.
```

El **tipo** indica qué clase de dato guardará la variable: un número, un texto... Los verás con calma en el capítulo 3. De momento te basta con saber que **number** significa número.

Por convenio, se suele usar lowerCamelCase para el nombre de las variables: los identificadores se escribirán con minúsculas y si hay varias palabras la primera letra de cada una irá en mayúsculas (excepto la letra inicial del nombre de la variable).

```
let litrosAguaRestante: number = 5;  
  
// Esta variable no tiene un valor inicial
```

```
let diasComida: number;

// Puedes cambiar el valor de cualquier variable una vez declarada.

litrosAguaRestante = 4;
diasComida = 2;
```

Además del convenio, hay unas cuantas reglas que no son opcionales. Si te las saltas, TypeScript no te dejará continuar:

- El nombre no puede empezar por un número: `2litros` no vale, `litros2` sí.
- No puede contener espacios ni guiones: `litros agua` y `litros-agua` no valen.
- No puedes usar palabras reservadas del lenguaje, como `let`, `const`, `if` o `class`.
- Se distinguen mayúsculas y minúsculas: `agua` y `Agua` son dos variables distintas. Mejor no tener las dos.

Constantes

Imagina que escribes el nombre de un objeto, sobre éste, utilizando un bolígrafo permanente. Una vez que nombras el objeto, ese nombre y ese contenido no cambiarán durante toda la expedición. Así, evitas confusiones y te aseguras de que lo importante siempre esté bien identificado. Pues algo similar son las constantes, una vez inicializadas ya no podrás modificar su valor durante todo el tiempo de vida de la aplicación.

El formato para las constantes es muy parecido al que usamos para declarar las variables, solo que en este caso es obligatorio asignar un valor inicial:

```
const NOMBRE_CONSTANTE: tipo = valor;
```

El nombre de las constantes se escribe en mayúsculas, con `_` para separar las palabras. Así, al leer el código, distingues de un vistazo los valores fijos que marcan las reglas de tu programa (el consumo diario de agua, la capacidad de la mochila) de los que van cambiando:

```
const LITROS_POR_DIA = 2;
const MENSAJE_AYUDA = "SOS";

// Si intentamos cambiar el valor de una constante, obtendremos un error.
// LITROS_POR_DIA = 3;
// Error de tipos: Cannot assign to 'LITROS_POR_DIA' because it is a
// constant.
```

**Nota en la cabaña**

En realidad, `const` tiene dos usos. El primero es el que acabas de ver: valores globales de configuración, que se fijan al principio del programa y afectan a todo él. Se escriben en MAYÚSCULAS. El segundo son valores que se calculan en un momento concreto y después ya no cambian, como los días que te durará el agua. Muchos equipos también los declaran con `const`, pero en lowerCamelCase: `const diasConAgua = litrosAgua / LITROS_POR_DIA; .`

Para no liarte con esto al principio, en este bloque usaremos `let` para todo lo que no sea un valor global de configuración, y `const` en MAYÚSCULAS solo para esos valores. Cuando empieces a escribir funciones, verás el segundo uso de `const`.

**Consejo de superviviente**

Usa solo caracteres del alfabeto inglés para los nombres de todo lo que uses en tu programa: variables, constantes, funciones, enumerados, clases, etc. Evita tildes, eñes y otros caracteres especiales (ñ, á, ü, etc.). Aunque algunos lenguajes los aceptan, pueden causar problemas de compatibilidad cuando compartas tu código con otros supervivientes o cuando uses herramientas de diferentes sistemas. Es como usar señales universales en lugar de dialectos locales: todos podrán entenderte sin importar de dónde vengas.

**Nota en la cabaña**

También existe la palabra clave `var` para declarar variables en JavaScript. Sin embargo, su uso no se recomienda porque puede provocar comportamientos inesperados relacionados con el ámbito y la redeclaración de variables. Siempre que puedas, utiliza `let` o `const` para mayor seguridad y claridad en tu código.

Buenas prácticas para un código limpio

Así como en una expedición, la organización y la claridad pueden marcar la diferencia entre sobrevivir y perderse, en programación seguir buenas prácticas te ayudará a mantener tu código legible y funcional:

- Nombra claramente tus variables y constantes. Usa nombres descriptivos y evita

abreviaturas confusas.

- Mantén la coherencia en el estilo de nombres: lowerCamelCase para variables y MAYÚSCULAS_CON_GUIONES para constantes.
- No reutilices variables para propósitos distintos, cada etiqueta debe tener un solo uso. Es decir, `díasConAgua` no debería pasar a guardar información del número de baterías.

Un ejemplo, de nombres muy poco descriptivos sería el siguiente:

```
let l = 2;  
let r = 4;  
let c = r / l;
```

Lo anterior, puede significar cualquier cosa y te hará perder un tiempo valioso para entenderlo. Sin embargo, el siguiente código es comprensible y te permite saber rápidamente qué representa cada variable:

```
// LITROS_POR_DIA y litrosAguaRestante son los de los ejemplos anteriores.  
let diasConAgua: number = litrosAguaRestante / LITROS_POR_DIA;
```

Evita los números mágicos

Cuando escribes código, a veces aparecen números que funcionan perfectamente pero cuyo significado no es evidente. Por ejemplo, imagina que calculas el peso de tu mochila así: `let pesoTotal = 8.2 + 3.5 + 1.8;`. Funciona, pero si regresas a ese código días después, ¿recordarás qué representa cada número? ¿Son kilogramos de agua, comida, munición? ¿Por qué esos valores específicos?

Estos valores literales sin contexto se conocen como **números mágicos**: números que aparecen directamente en el código sin explicación de su significado. Dificultan la lectura y el mantenimiento porque quien lee el código (incluyéndote a ti mismo en el futuro) debe adivinar qué representan.

Observa estos dos ejemplos del mismo cálculo:

Código con números mágicos:

```
// En un programa real, pesoMochila llegaría de otros cálculos.  
let pesoMochila: number = 11.5;  
let capacidadRestante = 15 - pesoMochila;
```

```
// Las comillas invertidas permiten meter valores en el texto (capítulo 3).  
console.log(`Puedes cargar ${capacidadRestante}kg más`);
```

Código sin números mágicos:

```
const CAPACIDAD_MAXIMA_MOCHILA_KG = 15;  
  
let capacidadRestanteKg = CAPACIDAD_MAXIMA_MOCHILA_KG - pesoMochila;  
  
console.log(`Puedes cargar ${capacidadRestanteKg}kg más`);
```

La diferencia es clara: en el segundo caso, cuando vuelvas a leer tu código comprenderás inmediatamente qué representa el valor que antes era 15. Además, si necesitas cambiar la capacidad de la mochila, solo modificas el valor de la constante en un lugar, y todos los cálculos que la usen se actualizarán automáticamente.



Consejo de superviviente

No todos los números son números mágicos. Valores obviamente comprensibles como 0, 1, 2 (para duplicar), 100 (para porcentajes), o -1 (para indicar “no encontrado”) suelen ser claros por sí mismos. Usa tu criterio: si un número necesita contexto para entenderse, conviértelo en una constante con nombre descriptivo.

Ejercicios de supervivencia

Ejercicio 2.1: Organizando tu inventario

Los gruñidos se escuchan más cerca ahora, y sabes que no puedes permitirte ningún error. Abres la mochila sobre la mesa con manos temblorosas y observas que tiene un espacio definido y un peso máximo que no se puede superar sin romperla. Pero dentro de esos márgenes críticos, sabes que a futuro tu inventario siempre cambiará: un día llevarás más agua, otro menos comida, y el peso variará con cada objeto que añadas o consumas mientras huyes de las hordas zombis.

En tu libreta de campo, con la linterna iluminando las páginas, decides registrar qué cosas son inmutables y cuáles cambian con el tiempo. Así sabrás en todo momento cómo organizar tus recursos para no quedarte indefenso en medio de la huida cuando la horda aparezca.

Tu misión: declara variables y constantes para organizar tu inventario de supervivencia.

- Define constantes con:
 - La capacidad de la mochila que será de 40 litros
 - El peso máximo de la mochila, que será 15 kilogramos
- Declara variables para recursos que pueden cambiar:
 - El número de litros de agua con un valor inicial de 3.5 litros.
 - El número de raciones de comida actuales y luego asígnele un valor de 12 raciones.
 - El peso actual con un valor inicial de 8.2 kilogramos.

Capítulo 3. Tipos de datos primitivos. Separando los suministros

No todos los suministros son iguales. Algunos son líquidos, otros sólidos; unos sirven para alimentarte, otros para orientarte. En programación ocurre lo mismo: cada dato tiene un **tipo**, y como superviviente, debes conocerlos tan bien como tu brújula o tu cuchillo.

Para indicar el tipo de un dato, se escribe `: tipo` justo después del nombre. Por ejemplo:

```
let zombis: number = 14;
let nombre: string = "Marcos";
```

Una vez que una variable tiene tipo, TypeScript vigila que no le metas otra cosa. Si lo intentas, el detector de metales pita:

```
let zombisDetectados: number = 14;
// zombisDetectados = "catorce";
// Error de tipos: Type 'string' is not assignable to type 'number'.
```

Los mensajes de error están en inglés y cuesta un poco leerlos al principio. En el apéndice A tienes los que más vas a ver, explicados con ejemplos.

No siempre hace falta escribir el tipo. Si das un valor inicial, TypeScript lo deduce solo. A esto se le llama **inferencia de tipos**:

```
let refugiosVisitados = 3; // TypeScript deduce que es number.
// refugiosVisitados = "tres";
// Error de tipos: Type 'string' is not assignable to type 'number'.
```

En este libro escribiremos el tipo casi siempre, para que lo tengas delante mientras aprendes. En tu código puedes omitirlo cuando el valor inicial ya lo deja claro.

Tipos numéricos

Los tipos de datos numéricos representan tus recursos cuantificables. Te dicen cuánto tienes, cuánto necesitas y cuánto puedes arriesgar.

Los **números enteros** son ideales para contar objetos, pasos o zombis.

```
// Número exacto. No guardas baterías partidas  
let baterias: number = 24;
```

Los **números decimales** son cruciales para medidas más precisas: distancia, tiempo, temperatura, peso.

```
// Aquí la precisión es vital.  
let temperatura: number = -15.7;
```



Consejo de superviviente

En los códigos de ejemplo verás que se usa el punto (.) para los valores decimales, así que tendremos 2.5 en lugar de 2,5 . Esto es debido a que la mayoría de lenguajes de programación (incluido TypeScript) reconocen el punto como separador decimal estándar.

Aunque aquí se ha explicado este tipo de datos por separado, en TypeScript no hay distinción para los valores enteros y reales. Es por ello que sólo existe el tipo `number` para representar ambos.

Como `number` sirve para los dos, TypeScript no te impedirá guardar 2.5 baterías. Decidir si un dato admite decimales es cosa tuya.

Hay otra sorpresa con los decimales. Suma dos cantidades de agua:

```
let aguaBotella: number = 0.1;  
let aguaCantimplora: number = 0.2;  
console.log(aguaBotella + aguaCantimplora); // 0.30000000000000004
```

Eso no es un fallo de TypeScript. Casi todos los lenguajes guardan los decimales en binario, y algunos, como 0.1 , no se pueden representar de forma exacta. Para mostrar el resultado, redondea con `toFixed()` indicando cuántos decimales quieres (lo verás a fondo en el capítulo 8):

```
console.log((aguaBotella + aguaCantimplora).toFixed(2)); // "0.30"
```



Nota en la cabaña

Al operar con valores numéricos, puede aparecer un valor misterioso: `NaN` (Not a Number). Es como encontrar una lata en tu mochila que parece comida, pero al abrirla... ¡está vacía o tiene algo incomible! `NaN` surge cuando intentas hacer operaciones matemáticas imposibles, como dividir 0 entre 0, o convertir una palabra en número. Si ves `NaN`, no confíes en ese recurso: es señal de que algo salió mal en tus cuentas y podrías quedarte sin provisiones cuando más lo necesitas. Revisa siempre tus cálculos y asegúrate de que no haya latas vacías en tu inventario.

Cadenas

Las cadenas de texto son como los mensajes que dejas escritos en las paredes o los datos que apuntas en tu libreta de supervivencia. Una cadena (`string`) representa un conjunto de caracteres: desde una simple palabra hasta párrafos completos de información crítica.

En el mundo post-apocalíptico, estas cadenas pueden contener:

- **Nombres de lugares:** “Refugio Norte”, “Hospital Abandonado”
- **Mensajes de emergencia:** “SOS - Necesitamos suministros médicos”
- **Coordenadas:** “Grid 47-B, Sector Industrial”
- **Inventarios:** “3x Latas, 2x Medicinas, 1x Radio”

```
let nombreDestinoSeguro: string = "Presa del Tablillas";  
let mensaje: string = "Grupo hostil avistado al sur";  
let coordenadas: string = "Lat: 40.7128, Lon: -74.0060";
```

Las cadenas son vitales para comunicarte con más supervivientes y registrar información que puede salvar vidas. Sin ellas, perderías la capacidad de documentar tus hallazgos o transmitir ubicaciones precisas por radio.

En TypeScript, hay tres formas de escribir una cadena:

```
// Usando comillas simples  
let advertencia: string = '¡Cuidado! Hay zombis cerca.';  
  
// Usando comillas dobles  
let aviso: string = "Refugio seguro encontrado";
```

```
// Usando template literals (plantillas de cadena) con el símbolo `.  
// Esto nos permite embeber variables, constantes y otro código mediante  
// el uso del siguiente formato: ${códigoAIncluir}  
let zona: string = "Centro Comercial";  
let sector: number = 7;  
let informe: string = `Explorando ${zona}, sector ${sector}`;
```

Las template literals también pueden ocupar varias líneas. Con comillas normales, para saltar de línea tienes que escribir `\n` :

```
let partePatrulla: string = `Zona: norte  
Zombis: 3  
Estado: controlado`;  
  
let parteRadio: string = "Zona: norte\nZombis: 3\nEstado: controlado";  
// Las dos variables contienen exactamente el mismo texto.
```



Consejo de superviviente

En la mayoría de lenguajes las cadenas van entre comillas. Asegúrate de cerrar bien dichas comillas, ya que si olvidas una, tu mensaje puede perderse en la jungla del código.



Nota en la cabaña

A lo largo de la guía, por compatibilidad con otros lenguajes, usamos comillas dobles `"` cuando no vayamos a incluir el valor de una variable o constante.

Booleanos

El tipo `boolean` consta sólo de dos valores **verdadero** (`true`) y **falso** (`false`). Esto te permite tomar decisiones. ¿Marchar o quedarse? ¿Encender el fuego o esperar?

```
let esZonaSegura: boolean = true;  
let tormentaAproximandose: boolean = false;
```

**Consejo de superviviente**

Nunca asumas que algo es verdadero por defecto. Verifica tus condiciones, o podrías dar por segura una zona repleta de trampas.

Cuando no tenemos recursos: `null` y `undefined`**`null`**

En la supervivencia, hay momentos en los que buscas en tu mochila y... ¡no queda nada! Ni agua, ni comida, ni munición. En programación, esos momentos se representan con `null`.

Tener un valor `null` es como mirar dentro de la mochila y ver que el compartimento está vacío, pero sabes que existe. Has guardado algo ahí antes, pero ahora no hay nada. Es una ausencia intencionada: tú mismo has decidido dejarlo vacío.

```
// Has decidido que el bolsillo de las pilas se queda vacío.  
let bolsilloPilas = null;  
console.log(bolsilloPilas); // null
```

`undefined`

`undefined` representa la ausencia de valor porque la variable no ha sido inicializada. Es asignado por el motor de JavaScript cuando declaras una variable sin darle valor, o cuando accedes a una propiedad inexistente. Aunque otros lenguajes tienen conceptos similares, la forma en que `undefined` se comporta es propia de JavaScript, así que es importante conocer sus peculiaridades.

```
// Declaras la variable, pero todavía no le has dado ningún valor.  
let proximoDestino;  
console.log(proximoDestino); // undefined
```

**Consejo de superviviente**

Antes de salir a explorar, revisa bien tu mochila y asegúrate de que no tienes compartimentos vacíos (`null`) ni olvidados (`undefined`). Si intentas usar algo que no existe, podrías encontrarte en serios problemas cuando más lo necesites.

**Nota en la cabaña**

Recuerda que `null` y `undefined` son valores distintos. Usa `null` cuando quieras indicar que algo está vacío a propósito, y evita usar `undefined` ya que el propio lenguaje lo “asignará automáticamente” cuando una variable o constante ni siquiera se hayan inicializado. ¡No confundas la falta de recursos con el olvido de prepararlos!

Unión de tipos

Enciendes la radio y solo se oye estática. Quieres guardar el último mensaje recibido, pero todavía no ha llegado ninguno. Una variable de tipo `string` no admite `null`, así que tienes que avisar a TypeScript de que puede estar vacía. Para eso se usa una barra vertical (`|`), que se lee como «o»:

```
// Todavía no has recibido ningún mensaje por radio.
let ultimoMensajeRadio: string | null = null;

// Horas después, alguien responde.
ultimoMensajeRadio = "Aquí refugio norte, ¿alguien escucha?";
```

A esto se le llama **unión de tipos**: la variable acepta cualquiera de los tipos que separes con `|`. Volverás a encontrarte con las uniones más adelante.

El tipo `any`

Imagina que encuentras una caja cerrada durante una exploración. No sabes qué hay dentro: podría ser comida, medicinas, munición, o incluso algo peligroso. En TypeScript, existe un tipo especial llamado `any` que funciona como esa caja misteriosa: puede contener cualquier tipo de dato.

```
let cajaEncontrada: any = "Radio de emergencia";
console.log(`Contenido inicial: ${cajaEncontrada}`);

// Después descubres que también puede ser un número
cajaEncontrada = 42;
console.log(`Ahora contiene: ${cajaEncontrada} baterías`);

// O incluso un booleano
cajaEncontrada = true;
```

```
console.log(`¿Está funcional? ${cajaEncontrada}`);
```

El tipo `any` es útil cuando:

- Mientras estás trabajando con código antiguo que no tiene tipos definidos.
- Recibes datos de fuentes externas (como mensajes de radio) donde no sabes qué esperar.
- Necesitas una solución temporal mientras determinas el tipo exacto.

Aunque exista este tipo de datos, debes usarlo con mucha precaución. Es como llevar una caja sin etiquetar en tu mochila: funciona, pero pierdes la ventaja de saber exactamente qué tienes. TypeScript no podrá ayudarte a detectar errores si usas `any` en exceso. Siempre que puedas, especifica el tipo de tus datos: tu yo del futuro te lo agradecerá.



Nota en la cabaña

Cuando encuentres datos misteriosos en tu código, puedes usar el operador `typeof` como una linterna para inspeccionar qué tipo de suministro tienes realmente. Es especialmente útil cuando trabajas con variables de tipo `any` o cuando recibes datos de fuentes externas.

```
let cajaDesconocida: any = "Medicinas";
console.log(typeof cajaDesconocida); // "string"

cajaDesconocida = 25;
console.log(typeof cajaDesconocida); // "number"

cajaDesconocida = true;
console.log(typeof cajaDesconocida); // "boolean"

cajaDesconocida = null;
// ¡Cuidado con esta trampa!
console.log(typeof cajaDesconocida); // "object"
```

El operador `typeof` devuelve una cadena que identifica el tipo: `"string"`, `"number"`, `"boolean"`, `"undefined"`, `"object"`, `"function"`, `"symbol"` o `"bigint"`. Ten en cuenta que `typeof null` devuelve `"object"` por razones históricas de JavaScript, así que no te fíes solo de esta inspección cuando busques valores nulos.

El tipo `unknown`

Si de verdad no sabes qué tipo va a tener un dato, hay una alternativa más segura que `any`: `unknown`. Admite cualquier valor, igual que `any`, pero no te deja usarlo hasta que compruebes qué es:

```
let cajaSellada: unknown = "Radio de emergencia";

// let etiqueta = cajaSellada + " (revisada)";
// Error de tipos: 'cajaSellada' is of type 'unknown'.

if (typeof cajaSellada === "string") {
  // Aquí TypeScript ya sabe que es una cadena.
  console.log(`La caja contiene: ${cajaSellada}`);
}
```

Con `any` abres la caja y metes la mano sin mirar. Con `unknown` tienes que mirar dentro antes de tocar nada.



Nota en la cabaña

No busques estos tipos en todos los lenguajes. `undefined` es propio de JavaScript (y por tanto de TypeScript), y `unknown` solo existe en TypeScript. La mayoría de lenguajes se apañan con un único valor para decir “aquí no hay nada”, como `null` en Java, `None` en Python o `nil` en Go, y no distinguen entre algo vacío a propósito y algo que nadie ha llegado a rellenar.

Otros tipos primitivos

Existen otros dos tipos primitivos que no usarás en este libro: `bigint`, para números enteros enormes, y `symbol`, para crear identificadores únicos. Si algún día los necesitas, ya sabrás que existen.

Ejercicios de supervivencia

Ejercicio 3.1: Clasificando suministros

El primer refugio que has encontrado apenas es más que una caseta de mantenimiento olvidada en las afueras. No hay mucho espacio, pero es lo suficientemente sólido como para pasar la noche y evitar los grupos de zombis que caminan sin descanso.

Con tu linterna inspeccionas cada rincón: un poco de comida enlatada, una vieja radio que tal vez funcione, y huellas recientes que no sabes si son humanas o no.

En tu libreta de campo decides clasificar los datos más importantes: el nombre del refugio, la temperatura, si es seguro, y el último mensaje de radio recibido. Si no lo registras con claridad, podrías olvidar detalles vitales cuando llegue el momento de decidir.

Tu misión: crea variables con diferentes tipos de datos para tu inventario:

- Nombre del refugio: una cadena.
- Temperatura actual: un número decimal.
- ¿Es zona segura?: un booleano.
- Último mensaje de radio: una cadena, o `null` si no hay contacto. Declárala como `string | null` y empieza sin contacto.
- Muestra por consola el estado del refugio con los cuatro datos.
- Muestra también, con `typeof`, qué tipo tiene la temperatura.
- Por último, simula que llega un mensaje: guarda un texto en la variable de la radio y vuelve a mostrarla.

```
// Ejemplo de cómo empezar:  
let nombreRefugio: string = "Refugio Alpha";  
  
console.log("=== ESTADO DEL REFUGIO ===");
```

Capítulo 4. Operadores. Cómo leer las señales del entorno

Cuando estás en una situación extrema, tu cerebro analiza constantemente el entorno: calcula cuánto te queda, compara riesgos y combina señales para tomar decisiones. En programación, eso lo haces con **operadores**.

Éstos son símbolos que permiten **transformar y evaluar datos**: sumar, restar, comparar, analizar condiciones... En otras palabras, son como tus sentidos, tus conocimientos y tu juicio trabajando juntos para sobrevivir.

Operadores aritméticos

Cuando necesitas saber cuántos días puedes resistir, qué distancia te queda o si los suministros alcanzan para toda la semana, estás haciendo **operaciones aritméticas**.

```
let comidaTotal: number = 50; // raciones disponibles
let consumoDiario: number = 3; // raciones por día
let metrosTotalesDestino: number = 15000; // metros hasta el refugio
let metrosRecorridos: number = 8500; // metros ya caminados

let diasSupervivencia: number = comidaTotal / consumoDiario;
let distanciaRestante: number = metrosTotalesDestino - metrosRecorridos;

console.log(`Puedes sobrevivir ${diasSupervivencia} días`);
console.log(`Te quedan ${distanciaRestante} metros por recorrer`);
```

Si ejecutas este código, verás que puedes sobrevivir `16.666666666666668` días. Es correcto, pero nadie cuenta así los días. Como ya viste en el capítulo 3 con el agua, `toFixed()` te permite quedarte solo con los decimales que quieras: `diasSupervivencia.toFixed(1)` da `"16.7"`.

En la mayoría de lenguajes de programación estos son los operadores aritméticos básicos:

Oper.	Significado	Prioridad	Ejemplo
**	Potencia	Más alta	<code>deteccion = zombis ** nivelRuido</code>
*	Multiplicación	Alta	<code>energiaTotal = pasos * caloriasPorPaso</code>
/	División	Alta	<code>dias = raciones / consumoDiario</code>
%	Módulo (resto)	Alta	<code>sobran = latas % latasPorCaja</code>
+	Suma	Media	<code>total = litrosAgua + litrosTe</code>
-	Resta	Media	<code>faltan = objetivo - avance</code>

Oper.	Significado	Prioridad	Ejemplo
-------	-------------	-----------	---------

Los operadores con la misma prioridad se resuelven de izquierda a derecha.

El módulo (`%`) devuelve el resto de la división: `10 % 3` da `1` , porque 10 entre 3 da 3 y sobra 1. Es muy útil para saber, por ejemplo, cuántas latas quedan sueltas al llenar cajas. Ojo con los negativos: el resultado toma el signo del primer número, así que `-7 % 3` da `-1` , no `2` .

Cuando calcules tus recursos, recuerda que la prioridad de los operadores puede cambiar el resultado. Para evitarlo, usa paréntesis para priorizar las operaciones. Podrías pensar que tienes más comida o agua de la que realmente hay, y esa confusión puede ser fatal en una situación crítica. ¡No dejes que un error de cálculo te deje sin provisiones!

Por ejemplo:

- Esta operación `3 + 4 * 12` , da como resultado `51` :
 - Por prioridad, primero se resuelve la multiplicación `4 * 12 = 48` .
 - Luego se realiza la suma `48 + 3 = 51` .
- Al usar paréntesis `(3 + 4) * 12` , el resultado será `84` :
 - Los paréntesis dan prioridad a la suma `3 + 4 = 7` .
 - A continuación, se lleva a cabo la multiplicación `7 * 12 = 84` .

Operadores de cadena

Puedes usar el operador `+` cuando trabajas con cadenas de texto. En este contexto no realiza la operación de suma sino que une o **concatena** los valores:

```
let nombreSuperviviente: string = "Alex";
let mensajeSOS: string = "SOS enviado por " + nombreSuperviviente;
// Resultado: "SOS enviado por Alex"

let distancia: number = 500;
let reporte: string = "Refugio a " + distancia + " metros";
// Resultado: "Refugio a 500 metros"
```

Si mezclas números y cadenas con `+` , el número se convierte a texto:

```
let suministros: number = 15;
let estado: string = "Tengo " + suministros + " raciones";
// Resultado: "Tengo 15 raciones"

// ¡Cuidado con este error común!
let resultado = "5" + 3; // "53" (cadena), no 8 (número)
```



Nota en la cabaña

JavaScript también te deja usar `*`, `/` o `-` con cadenas: intenta convertir la cadena en número y operar. `"10" * 2` da `20`, pero `"diez" * 2` da el temido **NaN**. TypeScript no permite estas operaciones y te avisa con un error de tipos (*The left-hand side of an arithmetic operation must be of type 'any', 'number', 'bigint' or an enum type*). Si por algo usamos TypeScript es para controlar que usas los tipos correctos.



Nota en la cabaña

Para evitar confusiones, usa **template literals** (plantillas de cadena) cuando combines texto y variables: ``Tengo ${suministros} raciones``. Es más claro y menos propenso a errores que la concatenación.

Eso sí, a lo largo de la guía verás que, cuando el texto es muy largo, usamos la concatenación para permitir que pueda verse correctamente.

Operadores de comparación

Saber si estás mejor o peor que ayer, si tienes lo mínimo necesario, o si has llegado al límite, es parte del día a día de la supervivencia. Cada decisión cuenta: comparar distancias, recursos o niveles de amenaza puede significar la diferencia entre resistir una noche más o perderlo todo. Tu mente analiza constantemente: ¿hay más zombis que balas?, ¿la temperatura ha bajado desde el amanecer?, ¿el refugio sigue siendo seguro o ha quedado comprometido?

```
let temperaturaTarde: number = 8;
let horaLlegada: number = 19; // 19:00 horas
let horaPuestaSol: number = 18; // 18:00 horas
let racionesRestantes: number = 15;
```

```

let nivelCritico: number = 20;

// ¿Es seguro continuar?

let haceFrio: boolean = temperaturaTarde < 12;
let necesitoRefugio: boolean = horaLlegada >= horaPuestaSol;
let debesEnviarSocorro: boolean = racionesRestantes < nivelCritico;

console.log(`Hace frío: ${haceFrio}`);
console.log(`Necesito refugio: ${necesitoRefugio}`);
console.log(`Debes enviar socorro: ${debesEnviarSocorro}`);

```

Estos operadores devuelven **valores booleanos**: `true` o `false`.

Oper.	Significado	Ejemplo
<code>==</code>	Igual a (con conversión)	<code>clima == "despejado"</code>
<code>===</code>	Igual a (sin conversión)	<code>agua === 5</code>
<code>!=</code>	Distinto de (con conversión)	<code>zona != "segura"</code>
<code>!==</code>	Distinto de (sin conversión)	<code>estado !== "peligro"</code>
<code>></code>	Mayor que	<code>litrosAgua > 2</code>
<code><</code>	Menor que	<code>zombis < municion</code>
<code>>=</code>	Mayor o igual que	<code>horaActual >= horaPuestaSol</code>
<code><=</code>	Menor o igual que	<code>temperatura <= 0</code>



Consejo de superviviente

No confundas `=` con `===`. El primero guarda un valor y el segundo compara. Si escribes `zombis = 0` cuando querías comprobar `zombis === 0`, no estás preguntando nada: acabas de poner el contador a cero.

En JavaScript existen dos tipos de operadores de igualdad y desigualdad que funcionan de manera diferente. Como superviviente, necesitas conocer la diferencia para evitar errores.

Operadores con conversión automática (`==` y `!=`)

```

// Así se comporta JavaScript:
// "1" == 1      da true: la cadena "1" se convierte al número 1.
// "3" != 3      da false: se convierte y compara 3 != 3.
// 0 == false    da true: el valor false se convierte a 0.

```

Operadores estrictos (`===` y `!==`)

```
// Así se comporta JavaScript:  
// "1" === 1    da false: tipos diferentes (string y number).  
// 0 === false  da false: tipos diferentes (number y boolean).
```

Los operadores estrictos comparan tanto el **valor** como el **tipo** de datos, sin conversiones automáticas. Esto es más predecible y seguro.

Si te preguntas por qué los ejemplos van comentados, es por un buen motivo: TypeScript ni siquiera te dejará compilar. Si comparas una cadena con un número, te avisa de que esa comparación no tiene sentido (*This comparison appears to be unintentional because the types 'string' and 'number' have no overlap*). Aun así conviene que conozcas la diferencia, porque la verás en código JavaScript.



Nota en la cabaña

Como recomendación, usa siempre `===` y `!==` para evitar comparaciones inesperadas.

Operadores lógicos

Una vez que sabes comparar, lo natural es empezar a combinar esas comparaciones para tomar decisiones más complejas. Y es que rara vez una sola señal es suficiente para tomar decisiones. Usas múltiples factores: clima, energía, terreno... y los combinas mentalmente. Eso es lo que hacen los **operadores lógicos**.

- `!` (no): invierte el valor booleano. Es el de mayor prioridad. Ejemplo: `!zonaPeligrosa`.
- `&&` (y): se cumple si se cumplen las dos condiciones. Prioridad media. Ejemplo: `aguaDisponible && terrenoSeguro`.
- `||` (o): basta con que se cumpla una de las dos. Es el de menor prioridad. Ejemplo: `climaBueno || refugioCercano`.

```
// Estado inicial del campamento  
let interruptor: boolean = true;  
let climaBueno: boolean = false;  
let refugioCercano: boolean = true;  
  
// Al pulsar el interruptor cambiamos su estado. Eso lo podemos hacer con !
```

```

let luzEncendida: boolean = !interruptor;
// Hemos apagado la luz: ahora es false

// Podremos continuar la marcha si el clima es bueno o tenemos un refugio
// cercano. Si cualquiera de las dos se cumple nos sirve para seguir
// caminando.
// Lo siguiente dará true puesto que hay un refugio cercano
let continuarMarcha: boolean = climaBueno || refugioCercano;
// Continuar marcha (OR): true
console.log(`Continuar marcha (OR): ${continuarMarcha}`);

// Podremos continuar la marcha cuando el clima sea bueno y tengamos un
// refugio cercano. Sólo podremos continuar cuando las dos condiciones se
// cumplan.
// Lo siguiente dará false puesto que no hay clima bueno.
continuarMarcha = climaBueno && refugioCercano;
// Continuar marcha (AND): false
console.log(`Continuar marcha (AND): ${continuarMarcha}`);

```

Para entender cómo se comporta cada operación booleana, usamos una tabla de verdad. En ella se muestran todas las combinaciones posibles de entradas y el resultado que obtendremos en cada caso.

Tabla de la verdad de la operación AND

(solo devuelve true cuando ambas condiciones lo son)

Entrada 1	Entrada 2	Resultado
false	false	false
false	true	false
true	false	false
true	true	true

Tabla de la verdad de la operación OR

(devuelve true si al menos una condición se cumple)

Entrada 1	Entrada 2	Resultado
false	false	false
false	true	true
true	false	true
true	true	true

Tabla de la verdad de la operación NOT

(invierte el valor: lo que era true pasa a false, y viceversa)

Entrada	Resultado
false	true
true	false

Cuando trabajes con operadores lógicos, no siempre usarás valores booleanos. Puedes utilizar cualquier otro tipo de dato que terminará, o transformándose en un valor `false` (a estos tipos de datos los llamamos valores *falsy*), o en un valor `true` (en este caso les llamaremos valores *truthy*).

- Valores falsy: `null`, `undefined`, `false`, `NaN`, `0`, `-0`, `""` (cadena vacía).
- Valores truthy: `true`, cualquier objeto, cualquier array (aunque esté vacío), cualquier número distinto de 0, cualquier cadena no vacía y los valores `Infinity` y `-Infinity`.

Dicho esto, cuando no uses valores estrictamente booleanos en los operadores AND y OR, la operación realmente devolverá uno de esos valores:

AND

- Si ambos valores son truthy, devuelve el segundo.
- Si no, se devuelve el primer valor falsy que haya.

```
// 1 && "Zombi" da "Zombi": los dos son truthy, se devuelve el segundo.
// 0 && "Zombi" da 0: es el primer valor falsy.
// "Zombi" && 0 da 0: es el primer valor falsy.
// null && 0 da null: es el primer valor falsy.
```

OR

- Se devuelve el primer valor `truthy` encontrado.
- Si ambos valores son `falsy`, se devuelve el segundo.

```
// 1 || "Zombi" da 1: es el primer valor truthy.
// 0 || "Zombi" da "Zombi": es el primer valor truthy.
// "Zombi" || 0 da "Zombi": es el primer valor truthy.
// null || 0 da 0: los dos son falsy, se devuelve el último.
```

Igual que antes, estas líneas van comentadas porque TypeScript avisa de que operar con valores fijos como `1` o `"Zombi"` no tiene sentido: el resultado se sabe de antemano. En un programa real usarás variables.



Nota en la cabaña

El operador `!` por norma general invierte un valor booleano, pero puede actuar con otro tipo de valores:

```
// !"" da true: valor falsy negado.
// !"SOS" da false: valor truthy negado.
// !0 da true: valor falsy negado.
// !12 da false: valor truthy negado.
// !null da true: valor falsy negado.
```

Al igual que ocurre con los operadores aritméticos, debes tener en cuenta la prioridad de los operadores lógicos. Así que recuerda que `&&` se evalúa antes que `||`. Por lo que, en el caso de querer priorizar una operación sobre otra, deberás usar paréntesis para dejar claro el orden y evitar sorpresas en tus decisiones.

Y es que no es lo mismo:

```
let energiaSuficiente: boolean = false;
continuarMarcha = (climaBueno || refugioCercano) && energiaSuficiente;
```

que te permitirá continuar la marcha sólo cuando haya energía suficiente y se cumpla al menos una de las otras dos condiciones, que lo siguiente:

```
continuarMarcha = climaBueno || refugioCercano && energiaSuficiente;
```

en cuyo caso, te permitirá continuar la marcha cuando haya un clima bueno, o exista un refugio cerca y a su vez tengas energía suficiente. Ya que el operador `&&` tiene

preferencia sobre `||`.

Con la primera opción, siempre será necesario tener energía suficiente. Sin embargo, con la segunda opción, simplemente con que haya un clima adecuado podrías continuar la marcha aunque te hayas agotado.



Nota en la cabaña

En ES2020 se introdujo un operador lógico con un nombre un poco extraño: **operador de coalescencia nula** (nullish coalescing en inglés), representado por el símbolo `??`. Funciona como un OR, pero omitiendo la mayoría de los valores `falsy` y sólo teniendo en cuenta los valores `null` y `undefined`. De forma que si el primer elemento tiene uno de esos valores, se devolverá el valor que hay tras `??`:

```
let combustibleDisponible: number | null = 0;
console.log(combustibleDisponible ?? 50); // 0: no es null ni undefined.
console.log(combustibleDisponible || 50); // 50: el 0 es falsy.
```

Este operador te permite asegurarte de que sólo en el caso de que el primer valor no esté definido o tenga valor nulo se utilice el segundo valor. Es muy útil sobre todo con valores numéricos.

Operadores unarios

En situaciones límite, a veces necesitas tomar decisiones rápidas con un solo recurso: sumar una unidad de energía, invertir una señal, o comprobar si tienes algo en la mochila. Los **operadores unarios** te permiten modificar o consultar el valor de una variable usando un solo operando, como si fuera un gesto instintivo de supervivencia.

Los operadores de **incremento** (`++`) y/o **decremento** (`--`) te permiten sumar o restar uno a un valor. Es decir, te permiten dar un paso adelante o atrás en el contador.

```
let latasRefresco = 5;
latasRefresco++; // Has encontrado una lata más: ahora tienes 6
latasRefresco--; // Te has bebido una lata: ahora tienes 5
```

El operador de **negación** (`!`), que ya conoces de los operadores lógicos, también cuenta como unario.

Finalmente, el operador de **cambio de signo** (`-`) cambia un valor positivo a negativo y

viceversa.

```
let temperaturaExterior = 10;
// Cambiamos el signo: ahora es -10. ¡Cuidado con la hipotermia!
temperaturaExterior = -temperaturaExterior;
// Al aplicar el cambio de signo a -10, el valor vuelve a ser 10.
temperaturaExterior = -temperaturaExterior;
```

Operador	Significado	Ejemplo
++	Suma uno al valor. Incremento.	latasRefresco++
--	Resta uno al valor. Decremento.	latasRefresco--
!	Invierte el valor booleano	!linternaEncendida
-	Cambia el signo de un número	-temperatura

Usar operadores unarios te permite reaccionar rápido, pero hay que tener cautela si usas los operadores `++` y `--` como prefijos o sufijos cuando asignas los valores. Ya que si se antepone el operador, primero se hace la operación de incrementar o decrementar el valor y luego la asignación:

```
let latas: number = 6;
let latasRestantes: number = --latas;
// latas y latasRestantes tienen el valor 5
```

Sin embargo, si el operador se sitúa tras la variable, primero se realizará la asignación y luego se incrementará o decrementará el valor:

```
let balas: number = 6;
let balasRestantes: number = balas--;
// balasRestantes tiene el valor 6 y balas el valor 5.
```

Asignación compuesta

Muy a menudo actualizas una variable a partir de su propio valor: encuentras 3 raciones más, o gastas la mitad de las bengalas. Escribir `raciones = raciones + 3` funciona, pero repite el nombre de la variable. Los operadores de **asignación compuesta** lo abrevian: pones el operador justo antes del signo igual (`=`).

```
let raciones: number = 10;
raciones = raciones + 3; // Se simplifica a:
raciones += 3;
```

●	<code>let bengalas: number = 8;</code>
●	<code>bengalas = bengalas / 2; // Se simplifica a:</code>
	<code>bengalas /= 2;</code>
●	
●	<code>let esSeguro: boolean = true;</code>
	<code>let hayRefugio: boolean = false;</code>
●	<code>esSeguro = esSeguro && hayRefugio; // Se simplifica a:</code>
●	<code>esSeguro &= hayRefugio;</code>

Operador	Equivale a	Ejemplo
<code>+=</code>	<code>x = x + valor</code>	<code>raciones += 3</code>
<code>-=</code>	<code>x = x - valor</code>	<code>agua -= 2</code>
<code>*=</code>	<code>x = x * valor</code>	<code>bengalas *= 2</code>
<code>/=</code>	<code>x = x / valor</code>	<code>bengalas /= 2</code>
<code>%=</code>	<code>x = x % valor</code>	<code>latasSeltas %= 6</code>

Los verás constantemente a lo largo del libro, sobre todo dentro de los bucles del capítulo 6.



Nota en la cabaña

A partir de ES2021 existen también los operadores compuestos lógicos `&&=`, `||=` y `??=`, como el `&&=` del ejemplo.

Ejercicios de supervivencia

Ejercicio 4.1: Calculadora de recursos

Han pasado tres días desde que encontraste un refugio abandonado. Tus recursos se agotan y debes calcular si lo que tienes alcanzará para completar la misión.

Mientras organizas los suministros en el sótano del refugio, encuentras marcas en el suelo: hay algo enterrado cerca que podría cambiar tu suerte. Pero antes de agarrar la pala, necesitas saber exactamente cuánto tiempo puedes resistir con lo que tienes. La excavación completa te llevará ocho días, y no hay margen para errores de cálculo.

Tu misión: realiza los siguientes cálculos en tu vieja libreta de campo:

- Calcula para cuántos días te llega el agua y para cuántos la comida, cada recurso por separado.
- Calcula cuánta agua y cuánta comida necesitas en total para los ocho días de excavación.
- Calcula la diferencia entre lo que tienes y lo que necesitas de cada recurso. Si sale negativa, te falta.
- Calcula qué porcentaje de tu agua y de tu comida gastas cada día, con dos decimales.
- Muestra todos los resultados por consola.

```
// Estado inicial del campamento tras tres días de organización
let litrosAguaTotal: number = 45;
let racionesComidaTotal: number = 30;
let diasMision: number = 8;
let consumoAguaDiario: number = 2; // Litros al día.
let consumoComidaDiario: number = 1.5; // Raciones al día.

console.log("=== ANÁLISIS DE RECURSOS DEL REFUGIO ===");
```

Ejercicio 4.2: Sistema de alerta temprana

La tormenta llegó sin avisar. Mientras dormías en el refugio, el viento comenzó a aullar entre las grietas de las paredes, y la temperatura descendió bruscamente. Un fuerte golpe te despierta, debes averiguar qué lo ha producido y evaluar los riesgos inmediatamente.

Has instalado varios sensores alrededor del refugio durante los últimos días, y ahora tu supervivencia depende de interpretar correctamente esas señales. El generador ha empezado a fallar intermitentemente, el combustible escasea, y la visibilidad se ha reducido considerablemente. Cada decisión que tomes en las próximas horas podría determinar si puedes sobrevivir a esta noche.

Tu misión: define un sistema de alerta que compruebe si las lecturas superan los umbrales críticos:

- Crea una variable booleana para cada alerta:
 - Temperatura peligrosa: por debajo de -10 °C.
 - Viento extremo: más de 40 km/h.
 - Visibilidad crítica: menos de 100 metros.
 - Combustible en reserva: menos del 20 %.

- Horario de alto riesgo: a partir de las 21:00.
- Calcula si hay al menos una alerta activa.
- Calcula si hay que evacuar de inmediato: cuando la temperatura es peligrosa y el viento es extremo a la vez, o cuando el combustible está en reserva en horario de alto riesgo.
- Muestra por consola el valor de cada alerta y los dos resultados finales.

```
// Lecturas actuales de los sensores durante la tormenta
```

```
let temperatura: number = -12;
```

```
let vientoKmh: number = 35;
```

```
let visibilidadMetros: number = 150;
```

```
let nivelCombustible: number = 18;
```

```
let horaActual: number = 22;
```

```
console.log("=== SISTEMA DE ALERTAS DE EMERGENCIA ===");
```

Ejercicio 4.3: Lógica de supervivencia

La tormenta ha pasado, pero sus consecuencias se sienten en cada rincón del refugio. Durante la noche, el estrés del temporal ha puesto a prueba todos los sistemas: el generador funciona de forma intermitente y los suministros se agotan más rápido de lo previsto.

Esta mañana, mientras revisas los daños con tu libreta de campo en la mano, la radio capta una transmisión entrecortada. No sabes si proviene de supervivientes, de una trampa o de simples interferencias, pero el mensaje sugiere que alguien podría necesitar ayuda.

Tu misión: tu experiencia te ha enseñado que en situaciones así no puedes basarte en una sola señal. Por lo que necesitas evaluar múltiples condiciones simultáneamente:

- Calcula si puedes salir: el perímetro tiene que ser seguro y, además, el clima tiene que estar estable o la radio tiene que funcionar.
- Calcula si el refugio está en modo crítico: el generador no funciona o te quedan menos de 3 raciones.
- Calcula si el combustible basta para que el generador funcione mientras estás fuera, sabiendo cuántas horas dura el viaje y cuánto gasta por hora.
- Calcula si merece la pena investigar la señal. Tienen que cumplirse todas estas condiciones:

- Puedes salir.
 - El refugio no está en modo crítico.
 - El combustible basta.
 - La zona de la señal es segura o, si no lo es, llevas al menos 4 raciones para esconderte unos días.
- Muestra por consola los cuatro resultados.

```
// Condiciones actuales
let generadorFuncional: boolean = true;
let perimetroSeguro: boolean = true;
let climaEstable: boolean = false;
let comunicacionActiva: boolean = true;
let racionesDisponibles: number = 6;
let litrosCombustible: number = 9;
let zonaSenalSegura: boolean = false;

// Datos del viaje hasta el origen de la señal
let horasAusencia: number = 8;
let consumoGeneradorPorHora: number = 1; // Litros por hora.

console.log("=== ANÁLISIS DE SEÑAL DE AUXILIO ===");
```

 Pista

Recuerda las prioridades en los distintos operadores que uses, y si es necesario utiliza paréntesis para clarificar el orden.

Continuará...

Hasta aquí la demo. Has preparado la mochila, has aprendido a etiquetar lo que llevas dentro, a distinguir unos suministros de otros y a hacer cuentas con ellos. Con eso ya puedes tomar una decisión sencilla, pero todavía no sabes qué hacer cuando la respuesta depende de varias cosas a la vez, ni cómo repetir una rutina sin copiarla veinte veces. Eso es lo que puedes seguir aprendiendo antes de cerrar este primer bloque que has visto. Las condiciones y los bucles, y con ellos sales al exterior.

El libro completo sigue mucho más allá. En el segundo bloque formas un grupo y os refugiáis en una central eléctrica, y por el camino aprendes funciones, arrays, cadenas y diccionarios. En el tercero la central cae, hay que tomar una cárcel abandonada y defenderla, y la programación orientada a objetos es lo que hace falta para organizar a tanta gente. El cuarto bloque es el primer invierno en la cárcel: cómo reaccionar cuando algo falla, cómo coordinar tareas que tardan y se ejecutan a la vez, y cómo hacer simulacros antes de que llegue la horda. Cada bloque termina con un desafío, y al final hay dos apéndices de consulta, uno con los errores más frecuentes de TypeScript y otro sobre cómo aprender con una IA sin que te lleve por mal camino.

Si te ha gustado lo que has leído hasta ahora, puedes conseguir el libro completo:

- [Tapa blanda](#)
- [eBook](#)

Revisa las tiendas de Amazon de otros países para encontrarlo también.